

삼성청년 SW·AI아카데미

관통 프로젝트 08

• 목차

- DOM 조작과 Event
- Input Event
 - 입력 유효성 확인 실습
- Drag Event
 - 카드 순서 바꾸기 실습

• 목차

- Scroll Event
 - 스크롤 시 나타나는 요소 실습
 - 스크롤 진행률 표시 실습
 - Parallax Scroll 실습 1
 - Parallax Scroll 실습 2
- 참고
 - 스크롤을 활용한 배경색 변경 실습
 - 가로, 세로 스크롤 혼합 실습

DOM 조작과 Event



DOM 이란?

웹 페이지의 모든 내용을 JavaScript가 이해하고 조작할 수 있도록 만든 구조화된 모델

DOM 조작이란?

JS가 웹페이지의 뼈대(HTML)와 디자인(CSS)을 실시간으로
통제하는 핵심 기술

Event 란?

웹 페이지 내에서 발생하는 모든 사건을 말하며,
JavaScript가 이를 감지하여 우리가 원하는 특정 동작을
실행하도록 만드는 방아쇠 역할

DOM 조작과 Event 처리는

웹페이지를 단순한 정적인 문서가 아니라,

사용자와 끊임없이 대화하고 반응하는

살아있는 인터페이스로 만들어줌

DOM 조작과 Event 처리 활용

1. 즉각적인 피드백
 - 사용자와 상호 작용
 - 실시간 유효성 확인
2. 효율적인 정보 관리
 - 평소에는 숨겨 두었다가 필요할 때 보여줌
3. 시각적 매력과 몰입감
 - 사용자 서비스 이용 경험 개선

Input Event



Input Event

데이터를 입력할 때마다 발생하는 이벤트

유사한 입력 이벤트 Keyup

Input Event

- 요소의 value (값)이 변경될 때
- 붙여넣기에도 반응하는 유효성 검사 가능

Keyup Event

- 키보드 키가 눌렀다가 떴을 때
- 특정 키(예: Enter 키)를 눌렀을 때만 동작 수행

입력 유효성 확인 실습

최대 글자 개수를 넘어서면 사용자에게 알려주기

Input Event를 활용한 유효성 확인

```
<!-- HTML 부분 -->
<h1>텍스트 입력 제한 (20자)</h1>

<input type="text" class="text-input" id="textInput" placeholder="최대 20자">
<div class="counter" id="counter">0 / 20</div>
```

기본적인 스타일 정의 및 움직임 요소 추가

Input Event를 활용한 유효성 확인

```
body {  
  margin: 20px;  
  background: white;  
}  
  
.text-input {  
  width: 300px;  
  padding: 10px;  
  border: 2px solid black;  
}  
  
.counter {  
  margin-top: 10px;  
}
```

```
/* 20자를 넘어갔을 때의 스타일 */  
.text-input.error {  
  background-color: #ffcccc;  
  animation: shake 0.5s;  
}  
  
/* 움직임을 정의 이름은 shake */  
@keyframes shake {  
  0%, 100% { transform: translateX(0); }  
  25% { transform: translateX(-5px); }  
  75% { transform: translateX(5px); }  
}
```


CSS animation

JS를 사용하지 않고도 복잡한 움직임을 구현할 수 있게 해주는 CSS의 기능

시작부터 끝까지 애니메이션의 전반적인 규칙을 정의

주요 animation 속성들

- animation-name : 어떤 동작을 할지 (Keyframes 이름 지정)
- animation-duration : 몇 초 동안 진행할지
- animation-timing-function: 속도 변화 (느리게, 시작/끝 등)

CSS keyframes 작성 예시

CSS 애니메이션의 개별 동작 단계와 변화 지점을 정의하는 규칙
@keyframes라는 특별한 문법으로 정의

```
/* 애니메이션의 이름은 'pulse' 입니다. */
@keyframes pulse {
  0% {
    transform: scale(1); /* 0%: 시작 지점 (초기 상태) */
    opacity: 1;          /* 원래 크기 */
                        /* 불투명도 100% */
  }
  50% {
    transform: scale(1.2); /* 50%: 중간 지점 */
    opacity: 0.5;          /* 크기 1.2배 */
                        /* 반투명 */
  }
  100% {
    transform: scale(1); /* 100%: 종료 지점 */
    opacity: 1;          /* 다시 원래 크기로 돌아옵니다. */
  }
}
```

CSS transforms

요소를 이동, 회전, 크기 조절, 기울이기 등을 할 수 있게 해주는 CSS 속성

- transform: [함수]([값]); 형태로 사용

transforms에서 사용하는 주요 함수들

- translate(x, y): x축과 y축으로 이동
- rotate(각도): 회전
- scale(x, y): x축과 y축 배율 조절
- skew(x각도, y각도): x축과 y축 기준으로 기울이기

Input Event를 활용한 유효성 확인

JS를 통한 input Event 발생 시 DOM 조작 요소 정의

```
const textInput = document.querySelector('#textInput')
const counter = document.querySelector('#counter')
const maxLength = 20
textInput.addEventListener('input', function (e) {
  const currentLength = e.target.value.length
  if (currentLength > maxLength) {
    e.target.value = e.target.value.substring(0, maxLength) // 입력 제한
    e.target.classList.add('error') // 에러 효과 추가
    // 에러 효과 500ms 이후 제거
    setTimeout(() => {
      e.target.classList.remove('error')
    }, 500)
    return
  }
  // 글자 수 표시
  counter.textContent = `${currentLength} / ${maxLength}`
})
```


실행 결과

20자를 넘어가면 붉은 색으로 변하며 입력창이 좌우로 흔들리는 것을 확인 할 수 있다.

텍스트 입력 제한 (20자)

최대 20자

0 / 20

텍스트 입력 제한 (20자)

가나다라마바사아자차카타파하가나다라마배

20 / 20

이어서..



Drag Event



Drag Event

웹 페이지의 요소를 끌어서 이동시키는 과정에서 발생하는 이벤트
단순한 하나의 이벤트가 아닌 끌기 시작부터 끌기 종료까지
모든 단계에서 순서대로 발생하는 여러 개의 이벤트 그룹

Drag Event 중요한 조건

`draggable = "true"`

- 드래그 하려는 HTML 요소에서 이 속성을 추가해야 요소가 드래그될 수 있음
- 기본 값은 `false` 로 요소가 드래그 되지 않음을 확인할 수 있음

Drag Event 종류

요소에서 발생하는 Event (마우스로 드래그 중인 요소에서 발생)

- dragstart 이벤트
 - 사용자가 요소를 끌기 시작하는 순간 **끌리는 요소**에서 발생
- drag 이벤트
 - 사용자가 요소를 끌고 다니는 과정 동안 **끌리는 요소**에서 계속 발생
- dragend 이벤트
 - 사용자가 마우스 버튼을 놓거나 드래그가 끝나는 순간 **끌리는 요소**에서 발생

Drag Event 종류

드래그 영역에서 발생하는 Event (드래그 되는 요소 아래 위치한 영역에서 발생)

- dragenter 이벤트
 - 끌고 있는 요소가 대상 영역 위로 들어올 때 **해당 영역에서 발생**
- dragover 이벤트
 - 끌고 있는 요소가 대상 영역 위에 머물고 있는 동안 **해당 영역에서 발생**
 - 이 때, 기본 동작을 취소해야 drop이 가능 (마우스 포인터 모양이 다름)
- dragleave 이벤트
 - 끌고 있는 요소가 대상 영역 밖으로 벗어날 때 **해당 영역에서 발생**
- drop 이벤트
 - 끌고 있던 요소를 대상 영역 위에 놓는 순간 **해당 영역에서 발생**

Drag Event 발생 예시

사용자가 **아이콘(요소)**을 클릭하고 끌기 시작합니다.

- **아이콘**에서 dragstart 이벤트 발생.

사용자가 **아이콘**을 **휴지통(영역)** 위로 가져옵니다.

- **휴지통**에서 dragenter 이벤트 발생.

사용자가 **아이콘**을 **휴지통** 위에서 움직입니다.

- **휴지통**에서 dragover 이벤트가 지속적으로 발생.

사용자가 **휴지통** 위에서 마우스 버튼을 놓습니다.

- **휴지통**에서 drop 이벤트 발생. (실제 데이터 처리)
- **아이콘**에서 dragend 이벤트 발생. (드래그 행위 종료)

카드 순서 바꾸기 실습

드래그를 통한 카드 순서 바꾸기 실습 진행

Drag Event 실습

- 드래그를 통해 카드 위치 변경
- 드래그 되는 동안 들어갈 곳을 임시로 표시

Drag Event 실습

- Drag Event 시작과 끝에 대한 이벤트 추가
- 카드가 드래그 되고 있음을 스타일링 하기 위함

```
// 1. 드래그 시작 리스너
// 현재 드래그 중인 것을 표시하기 위함
container.addEventListener('dragstart', (e) => {
  draggedCard = e.target // 드래그 카드 저장
  e.target.classList.add('dragging')
})
```

```
// 3. 드래그 종료
// 드래그 중 표시를 제거하기 위함
container.addEventListener('dragend', (e) => {
  draggedCard = null // 드래그 카드 초기화
  e.target.classList.remove('dragging')
})
```

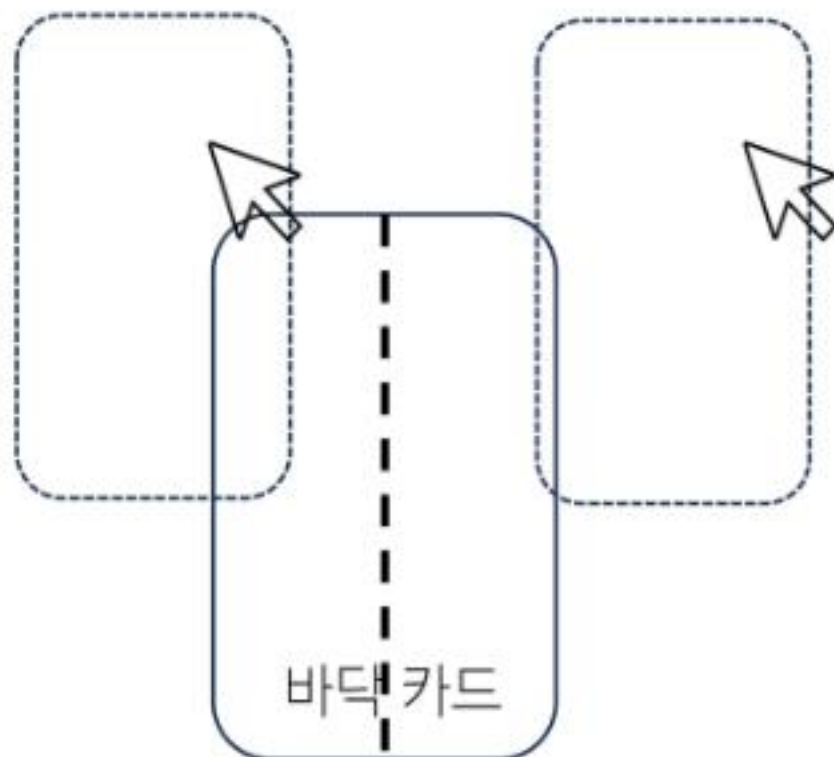
e.target : 실제 이벤트가 발생한 요소

e.currentTarget: 이벤트 리스너가 설정된 요소

Drag Event 실습

카드 배치 원리

- 현재 드래그 중인 마우스 포인터의 위치와 바닥의 카드의 중심의 위치를 계산하여
- 드래그 중인 마우스 포인터가 가운데 보다 앞쪽이면 드래그 카드는 바닥 카드 앞에 위치
- 드래그 중인 마우스 포인터가 가운데 보다 뒤쪽이면 드래그 카드는 바닥 카드 뒤에 위치



Drag Event 실습

주요 코드

```
container.addEventListener('dragover', (e) => {  
  e.preventDefault()  
  ...  
})
```

Drag 이벤트에서 `preventDefault()` 를 사용해야 하는 이유

- 기본적으로 드래그 기본 동작이 드래그를 허용하지 않음 이기 때문에
마우스 커서가 드래그 금지 표시로 나타나게 됨

Drag Event 실습

주요 코드

```
const mouseX = e.clientX // 드래그 중인 마우스 X좌표
```

clientX

- 현재 마우스 위치 값이 담겨 있음
- Viewport 내의 수평 좌표

<https://developer.mozilla.org/ko/docs/Web/API/MouseEvent/clientX>

Drag Event 실습

주요 코드

```
otherCards.forEach(card => {  
  const rect = card.getBoundingClientRect()  
  ...  
})
```

getBoundingClientRect

- 요소의 크기와 Viewport에서의 상대적인 위치 정보를 반환

<https://developer.mozilla.org/ko/docs/Web/API/Element/getBoundingClientRect>

Drag Event 실습

주요 코드

```
// 카드의 중심 위치 계산
const cardCenter = rect.left + rect.width / 2
// 카드 중심과 드래그 중인 카드의 거리를 확인해서 가장 가까운 카드 찾기
const distance = Math.abs(mouseX - cardCenter)

if (distance < minDistance) {
  minDistance = distance
  closestCard = card           // 가장 가까운 카드 갱신
  closestCardCenter = cardCenter
}
```

가장 가까운 카드를 찾는 알고리즘

Drag Event 실습

주요 코드

```
// 드래그 중인 카드 위치 변경
if (closestCard) {
  if (mouseX < closestCardCenter) {
    container.insertBefore(draggedCard, closestCard)
  } else {
    container.insertBefore(draggedCard, closestCard.nextSibling)
  }
}
```

insertBefore

- 기준이 되는 노드의 앞에 새로운 노드를 추가할 때 사용

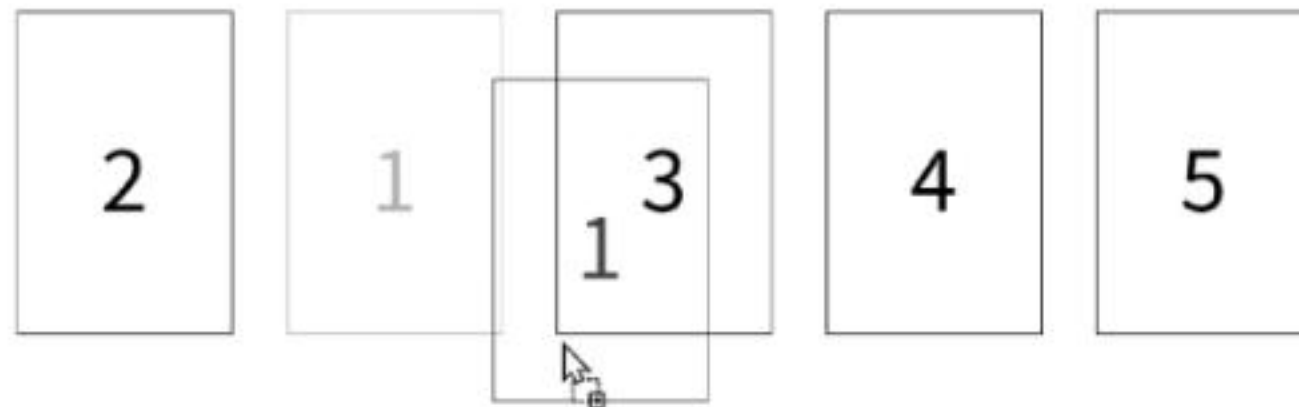
<https://developer.mozilla.org/en-US/docs/Web/API/Node/insertBefore>

실행 결과

카드를 드래그 하면 추가될 위치가 나타나며 이동시킬 수 있음을 확인

드래그 앤 드롭 카드 정렬

카드를 마우스로 드래그해서 순서를 바꿔보세요



이어서..



Scroll Event



Scroll Event

웹 페이지의 콘텐츠를 상하좌우로 이동(스크롤)시킬 때마다
지속적으로 발생하는 이벤트

Scroll Event 주요 속성

- `window.scrollY`
 - 브라우저 창의 수직 스크롤 위치
 - 권장하는 표준 속성
- `element.scrollHeight`
 - 요소에 포함된 전체 콘텐츠의 높이로 스크롤 되지 않은 부분까지 포함되며 총 길이를 파악하여 스크롤 막대의 끝에 도달했는지 확인할 때 사용

스크롤시 나타나는 요소 실습

스크롤시 나타나는 요소 실습

- 투명하게 설정된 요소의 상단 위치를 스크롤 이벤트가 발생할 때 마다 확인
- 해당 요소의 상단 위치가 특정 위치에 도달하게 되면 투명도를 JS로 조절하여 나타나게 만듦
- transition 속성을 활용 부드럽게 나타나도록 설정

스크롤 나타나는 요소 실습

주요 코드

```
// 스크롤 이벤트 처리
function checkScroll() {
  const windowHeight = window.innerHeight // 현재 창의 높이

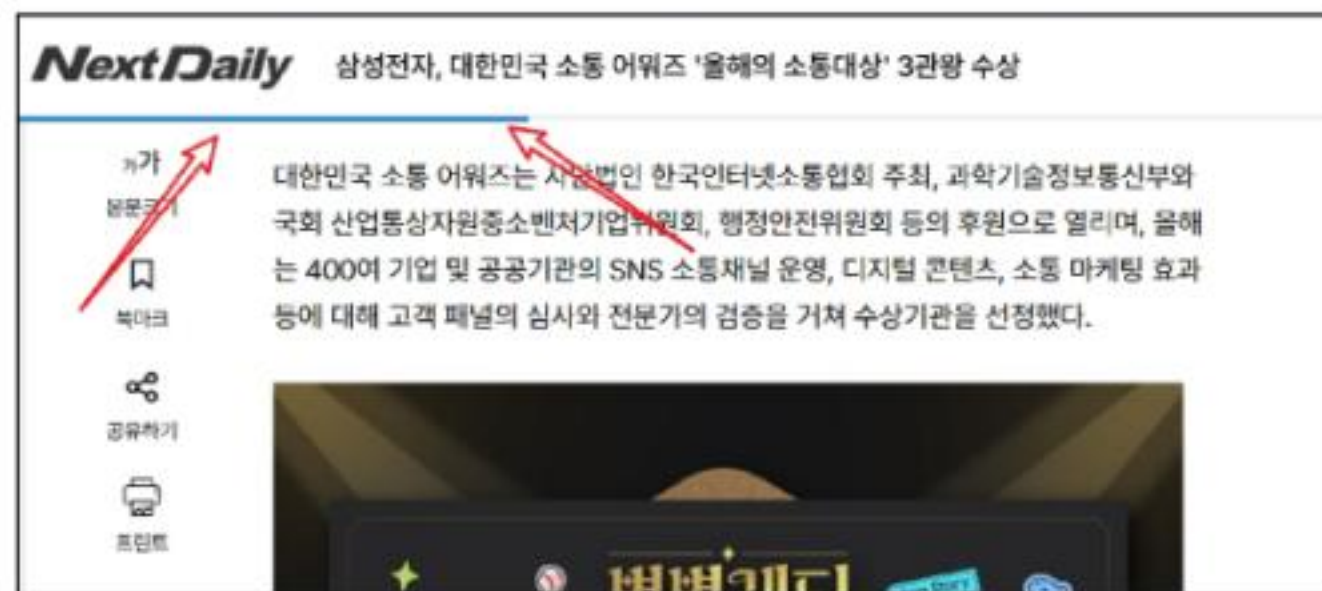
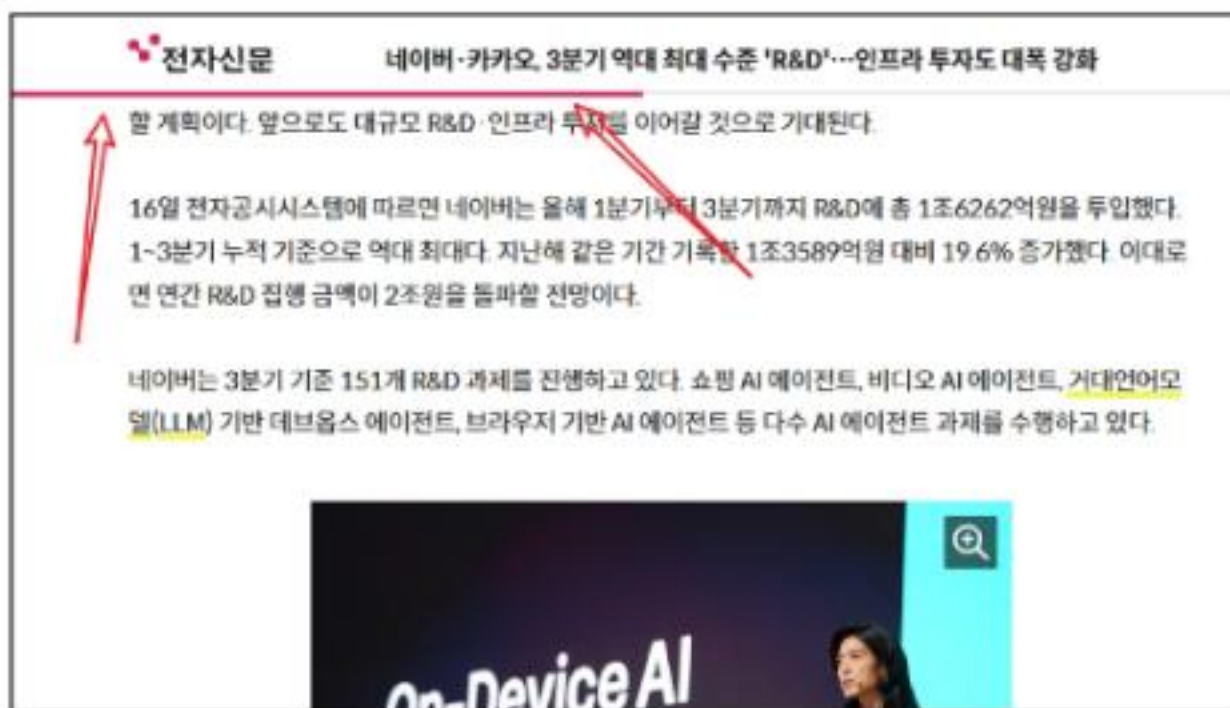
  cards.forEach(card => {
    const cardTop = card.getBoundingClientRect().top // 상단 정보 확인

    // 바로 나타나기 보다는 좀 더 위에서 나타날 수 있도록 200px 차이 확보
    if (cardTop < windowHeight - 200) {
      card.classList.add('visible')
    }
  })
}
```


스크롤 진행률 표시 실습

스크롤 진행도 표시 실습

- 화면 상단에 스크롤에 따른 진행률을 나타내는 서비스들이 존재
- 내용이 많아 스크롤이 길 때 사용가능
 - 주로 언론사 기사 페이지에서 활용



출처: <https://www.nextdaily.co.kr/news/articleView.html?idxno=244339>

출처: <https://www.etnews.com/20251114000298>

스크롤 진행도 표시 실습

주요 원리

- 진행률에 따라 프로그래스 바의 너비를 DOM 조작을 통해 갱신
- 스크롤 진행률 계산 방법
 - 현재 스크롤 높이 / 전체 높이 (X)
 - 처음에 보여지는 높이는 제외해야 스크롤 가능 범위가 됨
 - 스크롤 한다는 것은 보이지 않는 부분을 내려간다는 의미
 - 스크롤 이동 가능 범위 = 전체 높이 - Viewport 높이
 - **스크롤 진행률** = 현재 스크롤 위치 / 스크롤 이동 가능 범위



스크롤 진행도 표시 실습

주요 코드

```
function updateProgress (e) {  
  const scrollY = window.scrollY // 현재 스크롤 위치  
  // 스크롤 가능한 범위  
  const scrollHeight = document.documentElement.scrollHeight - window.innerHeight  
  
  const progress = scrollY / scrollHeight  
  
  progressFill.style.transform = `scaleX(${progress})`  
}  
  
window.addEventListener('scroll', updateProgress)
```

documentElement.scrollHeight

- 콘텐츠의 총 높이를 나타내며, 바깥으로 넘쳐서 보이지 않는 콘텐츠도 포함함

window.innerHeight

- 막대 높이를 포함한 창 내부 높이

<https://developer.mozilla.org/ko/docs/Web/API/Element/scrollHeight>

<https://developer.mozilla.org/ko/docs/Web/API/Window/innerHeight>

Parallax Scroll

Parallax Scroll

스크롤할 때 **화면의 배경과 앞에 보이는 내용이 서로 다른 속도로 움직여**, 화면에 깊이나 3D 같은 입체감을 주는 시각적 효과



Parallax Scroll 적용 예시

스크롤에 따른 입체감



스크롤에 따라 텍스트와 뒷 배경이 입체감 있게 변경되는 것을 볼 수 있음

출처: <https://dddhotel.jp/en/>

Parallax Scroll 적용 예시

스크롤에 따른 배경 및 이미지 전환



출처: <https://www.cyclemon.com/>

Parallax Scroll 원리 실습 1

Parallax Scroll 원리 실습 1

주요 코드

- 요소 별로 움직이는 속도를 설정

```
// 각 레이어의 속도 설정
const speeds = {
  back: 0.2,
  middle: 0.3,
  front: 0.4
}
```

Parallax Scroll 원리 실습 1

주요 코드

```
window.addEventListener('scroll', () => {  
  const scrolled = window.scrollY // 현재 스크롤 위치  
  
  // 각 레이어를 다른 속도로 이동 (음수로 위로 올라가게)  
  layerBack.style.transform = `translateY(${ -scrolled * speeds.back }px)`  
  layerMiddle.style.transform = `translateY(${ -scrolled * speeds.middle }px)`  
  layerFront.style.transform = `translateY(${ -scrolled * speeds.front }px)`  
})
```

- 스크롤을 내리면 위로 올라갈 수 있도록 음수로 지정
- 스크롤 위치와 속도를 곱하여 이동하는 거리를 요소별로 다르게 지정

Parallax Scroll 원리 실습 1

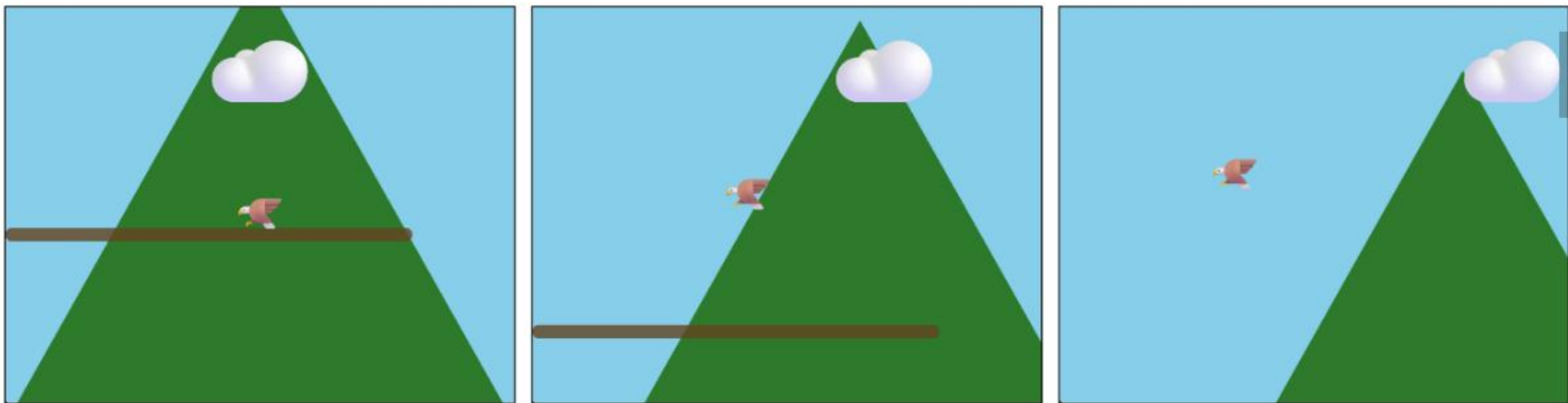
스크롤 전 / 후



Parallax Scroll 원리 실습 2

Parallax Scroll 원리 실습 2

앞서 적용한 기본 Parallax Scroll 원리를 활용하여 스크롤시 요소 별 동작을 부여하여 스크롤 했을 때 애니메이션 처럼 동작하도록 설정



Parallax Scroll 원리 실습 2

주요 코드

```
// 각 레이어의 속도 설정
const speeds = {
  mountain: 0.05,
  cloud: 0.1,
  branch: 0.1,
  bird: 0.4
}
```

```
window.addEventListener('scroll', () => {
  const scrolled = window.scrollY // 화면이 얼마나 내려왔는지 위치 확인

  // 각 레이어를 다른 속도로 이동
  layerMountain.style.transform = `translate(${scrolled * speeds.mountain * 1.5}px, ${scrolled * speeds.mountain}px)`
  layerCloud.style.transform = `translate(${scrolled * speeds.cloud}px)`
  // 나뭇가지는 조금씩 아래로 이동
  layerBranch.style.transform = `translateY(${scrolled * speeds.branch}px)`
  // 새는 좌상단 대각선으로 이동 (위로 + 왼쪽으로) translate(x, y) 활용
  layerBird.style.transform = `translate(${scrolled * speeds.bird * 0.1}px, ${scrolled * speeds.bird * 0.05}px)`
})
```

참고



Scroll Event 활용하기

앞서 실습한 내용을 바탕으로 보다 드라마틱한 페이지 연출이 가능함

- 스크롤에 따른 배경색 변경하는 실습
- 같은 방향의 스크롤임에도 가로, 세로 복합적으로 동작

스크롤을 활용한 배경색 변경 실습

스크롤을 활용한 배경색 변경 실습

동작 원리

- 스크롤 진행률은 0~1 의 범위를 가지고 있음
 - 스크롤 진행률 = 현재 스크롤 위치 / 스크롤 가능한 높이
- 이를 인덱스의 개수만큼 곱하고, 내림 계산을 하면 구간 별로 Index 를 다르게 가져갈 수 있음
 - 배열의 길이가 5 라면, 진행도에 배열의 길이 만큼 곱하고 내림을 하면 구간 별로 index가 계산됨

예시)

- 스크롤 진행률이 0.3 부근인 경우, 5를 곱하면 1.5가되고 내림 계산을 하면 1이 됨. 이 때 index는 1로 설정할 수 있음.
- 스크롤 진행률이 0.5 부근인 경우, 5를 곱하면 2.5가 되고 내림 계산을 하면 2가 됨. 이 때 index는 2로 설정할 수 있음.
- 스크롤 진행률이 0.8 부근인 경우, 5를 곱하면 4.0이 되고 내림 계산을 하면 4가 됨. 이 때 Index는 4로 설정할 수 있음.

스크롤을 활용한 배경색 변경 실습

실습 내용

스크롤에 따라 새벽, 아침, 낮, 저녁, 밤 총 5구간을 설정하여 배경색을 달리하고,
스크롤 진행도에 따라 해와 달의 움직임을 표현하는 실습

스크롤을 활용한 배경색 변경 실습

주요 코드 (배경 인덱스)

```
// 현재 스크롤 위치와 진행률 계산
const scrollY = window.scrollY
const scrollHeight = document.documentElement.scrollHeight - window.innerHeight
const progress = Math.min(scrollY / scrollHeight, 1) // 0~1 사이 값

// 진행률을 색상 인덱스로 변환
const colorIndex = Math.floor(progress * colors.length)
const currentColorIndex = Math.min(colorIndex, colors.length - 1)

// 정해진 색상 적용
const currentColor = colors[currentColorIndex]
```

진행률 계산에서 min 을 사용한 이유

- 모바일이나 특정 브라우저에서 높이보다 스크롤의 위치가 넘어가는 경우가 발생하여 진행률이 1 이상이 될 수 있음
- 0 ~ 1 사이의 값이 아닌 1 이상의 값이 나오는 경우가 발생하면 최대 Index 를 넘어버리게 되어 min 을 통해 제한

스크롤을 활용한 배경색 변경 실습

해의 이동 코드

```
// 해와 달의 위치 계산
function calculateCelestialPosition(progress, isNightTime = false) {
  const screenWidth = window.innerWidth

  if (!isNightTime) {
    // 해의 경로 (새벽 -> 낮 만 떠 있도록): 0% ~ 60% 진행률
    const normalizedProgress = Math.min(progress / 0.6, 1)
    const halfSunSize = sunSize / 2
    const x = -halfSunSize + normalizedProgress * (screenWidth - halfSunSize)

    // 저녁(60%) 이후에는 해를 숨김
    sun.style.opacity = progress >= 0.6 ? 0 : 1
    return { x }
  }
}
```

...

스크롤을 활용한 배경색 변경 실습

달의 이동 코드

```
// 해의 경로 코드 이어서
// 달의 경로 (저녁 -> 밤): 60% ~ 100% 진행률
if (progress < 0.6) {
    moon.style.opacity = 0
    return null
}
// 남은 달의 이동 구간은 해의 이동구간 0.6을 뺀 0.4 임
const normalizedProgress = (progress - 0.6) / 0.4 // 0~1 사이 값으로 정규화
const x = normalizedProgress * (screenWidth - moonSize)

moon.style.opacity = 1 // 달 표시
return { x }
}
```


심화 기능

- 배경색을 부드럽게 변경하기 위한 선형 보간 계산 추가
 - 선형 보간: 두 값 사이를 부드럽게 이어주는 계산 방법
 - 예: 빨강(255, 0, 0)과 파랑(0, 0, 255) 사이의 중간 색상 구하기
 - 시작값 + (끝값 - 시작값) * 진행률
- 해와 달을 곡선 형태로 표기하기 위한 포물선 곡선 계산 공식 추가
 - 기본 포물선 공식: $ax^2 + bx + c$
 - 수학적 원리:
 - $2 * \text{normalizedX} * (1 - \text{normalizedX})$ 포물선 공식
 - normalizedX는 0~1 사이의 값 (진행률)

실제 응용 서비스

- 연속된 이미지를 불러와서 스크롤 진행도에 따라 index를 변경하면 자연스럽게 동영상의 재생되는 효과를 낼 수 있음

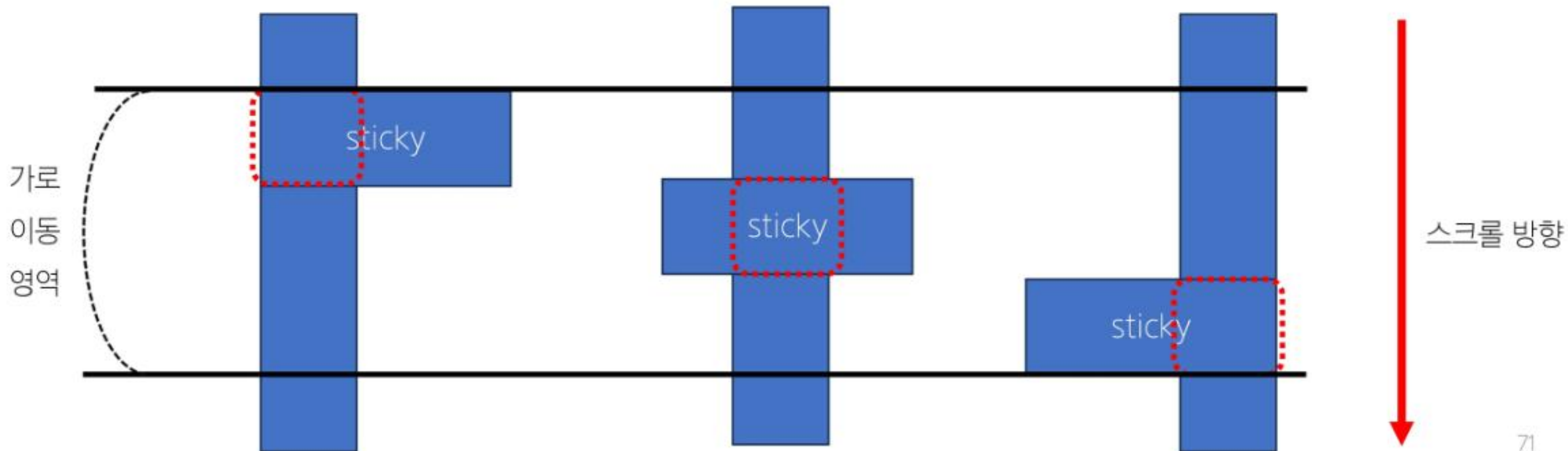


가로, 세로 스크롤 혼합 실습

가로, 세로 스크롤 혼합 실습

동작 원리

붉은 색 점선이 사용자에게 보여지는 화면일 때, 스크롤 내려감과 함께 가로 배치된 요소가 이동하면서 보여짐



가로, 세로 스크롤 혼합 실습

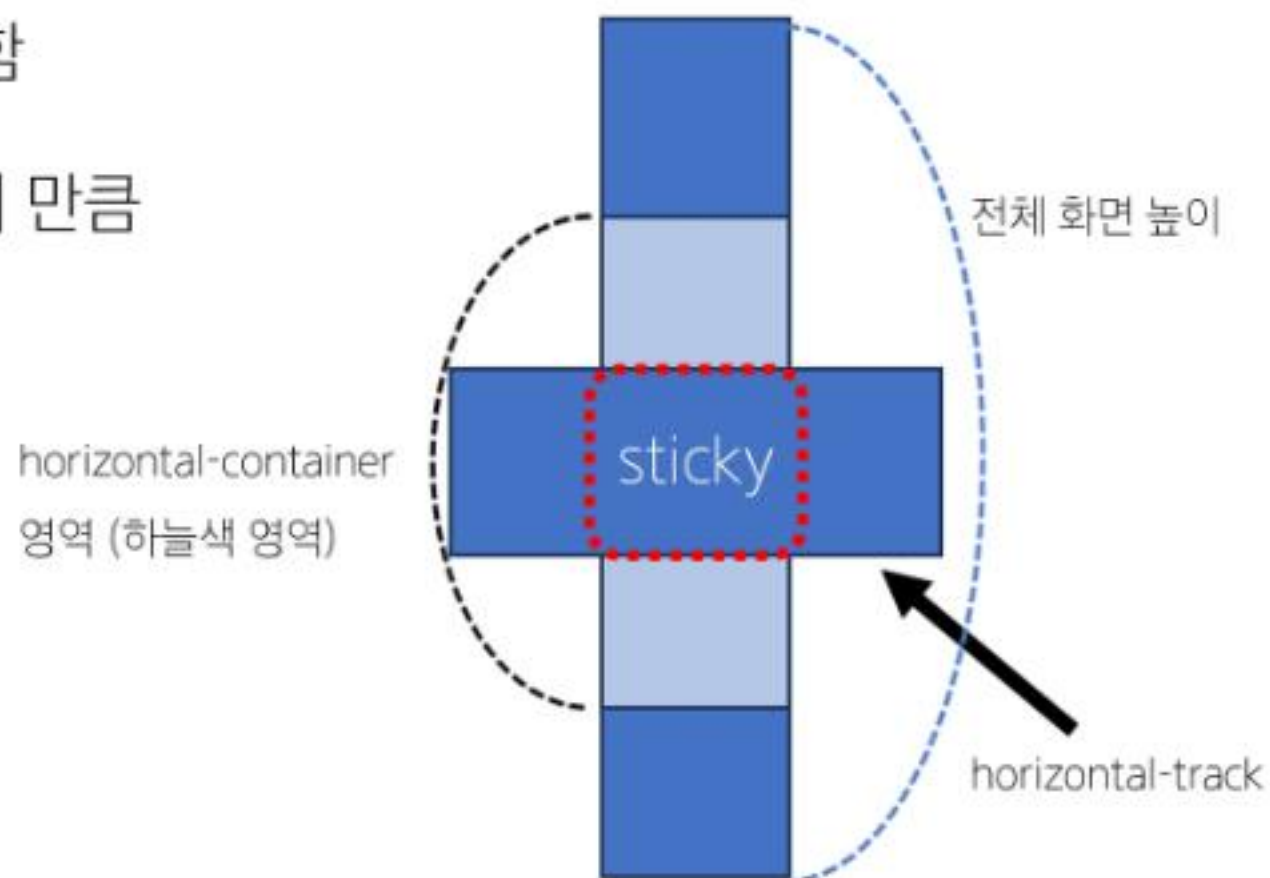
실습을 위해 필요한 CSS 속성

- 가로 스크롤하는 영역(horizontal-track)의 너비(width)와 실제 스크롤 (horizontal-container)되는 높이(height)가 같아야 함
 - horizontal-track의 너비는 개별 page의 개수로 동일하게 맞춤
- 가로 정렬을 위해 Flex 설정이 필요함
- 가로 정렬된 요소를 숨기기 위해 **overflow: hidden** 설정이 필요함
 - hidden을 하는 이유, 가로 스크롤바를 없애기 위함
- 가로 스크롤 하는 동안 viewport에 고정하기 위해 sticky 설정이 필요함
 - 하지 않으면 스크롤과 함께 올라가버림
 - fixed인 경우 가로 이동 후에도 고정되어 있게됨

가로, 세로 스크롤 혼합 실습

동작 원리

- 스크롤을 하다가 horizontal-container 영역에서 가로 스크롤이 동작할 영역
- 스크롤을 하게 되면 horizontal-container의 top의 위치가 이동함
- horizontal-container의 top의 위치가 스크롤되어 이동하는 거리 만큼 horizontal-track의 위치도 이동하여 화면에 출력하는 원리



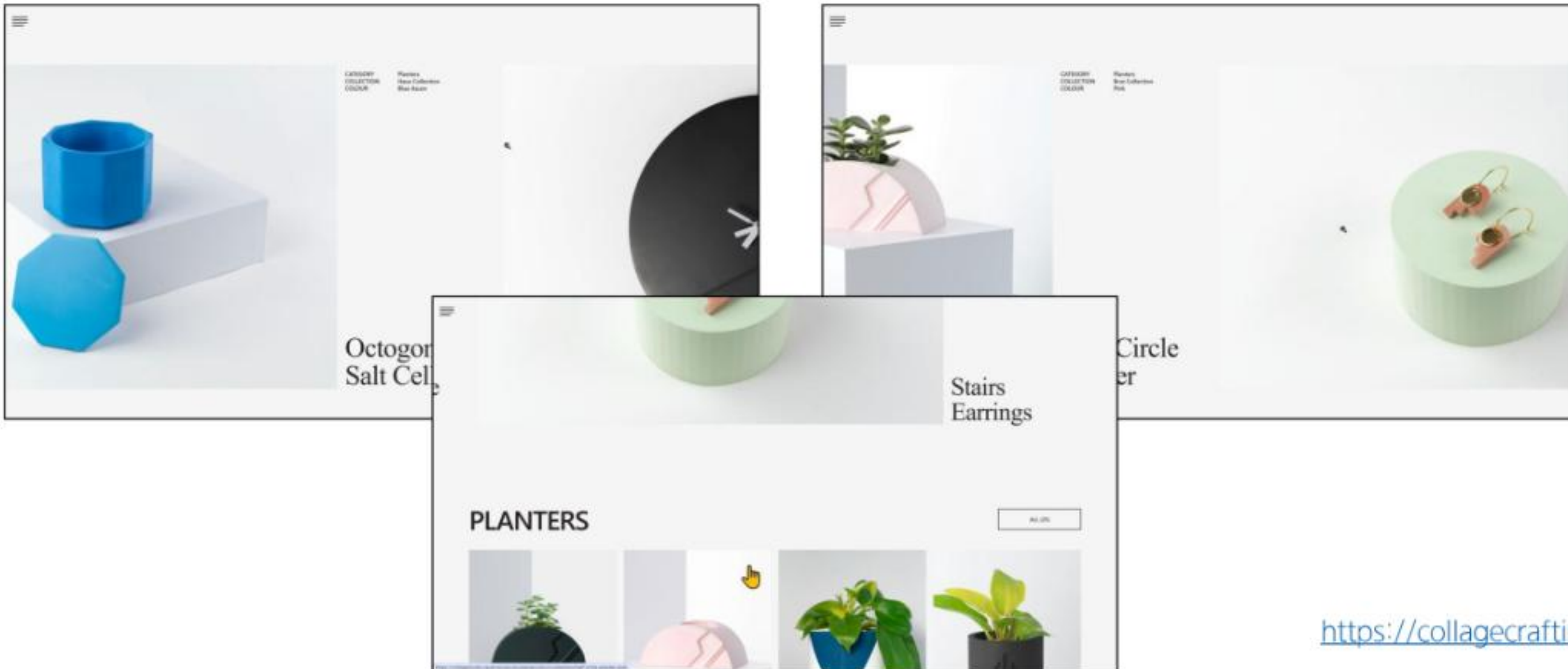
가로, 세로 스크롤 혼합 실습

주요 코드

```
function updateHorizontalPosition() {  
  // getBoundingClientRect: 요소의 화면상 위치 정보  
  const containerRect = horizontalContainer.getBoundingClientRect()  
  const containerHeight = horizontalContainer.offsetHeight // 컨테이너 전체 높이 (500vh)  
  const windowHeight = window.innerHeight // 브라우저 창 높이  
  
  // 가로 스크롤 영역(horizontal-container)이 화면에 보이는 동안만 실행  
  if (containerRect.top <= 0 && containerRect.bottom >= windowHeight) {  
    // 스크롤 진행률 계산 (0에서 1 사이 값)  
    const scrollProgress = Math.abs(containerRect.top) / (containerHeight - windowHeight)  
  
    // 가로 이동 거리 계산  
    const maxMove = 400 // 4페이지 이동 (페이지1은 고정, 2~5페이지 이동 = 400vw)  
    const moveDistance = scrollProgress * maxMove  
  
    // translateX로 가로 이동 적용 (-는 왼쪽으로 이동)  
    horizontalTrack.style.transform = `translateX(-${moveDistance}vw)`  
  }  
}
```

실제 적용 사례

PC 화면일 때 확인 가능



<https://collagecrafting.com/en>

관통 프로젝트 안내



다음 시간에
만나요!

삼성 청년 SW · AI 아카데미